Analyse lexicale

Dr. Aida Lahouij
Maitre assistante de l’enseignement supérieur et de la recherchce scientifique
Aida.lahouij@gmail.com

03 Analyse Lexicale Manuelle

3. Analyse Lexicale Manuelle

Les étapes générales pour implémenter manuellement l'automate de l'analyse lexcicale :

1. **Identifier les différents types de jetons** : La première étape consiste à identifier les différents types de jetons que l'on souhaite reconnaître dans le programme source, tels que des **identificateurs**, des **nombres**, des **opérateurs**, des **mots-clés**, etc.

2. **Définir les règles lexicales** : Pour chaque type de jeton, il est nécessaire de définir une ou plusieurs règles lexicales qui permettent de reconnaître le jeton dans le programme source. Les règles lexicales sont généralement définies sous forme **d'expressions régulières** qui décrivent les motifs à rechercher dans le texte.

3. Analyse Lexicale Manuelle

Les étapes générales pour implémenter manuellement l'automate de l'analyse lexicale :

3. **Implémenter les transitions de l'automate :** Les transitions sont des **fonctions** qui prennent en entrée **un état de l'automate** et **un caractère de la chaîne d'entrée**, et qui **renvoient l'état suivant de l'automate** en fonction du caractère.

4. **Stocker les jetons reconnus :** Enfin, lorsque l'automate de l'analyse lexicale reconnaît un jeton, il est stocké dans **une structure de données** telle qu'une liste ou une table de hachage pour une utilisation ultérieure par les étapes suivantes de la compilation.

3. Analyse Lexicale Manuelle

Rappel:

3. Analyse Lexicale Manuelle

Rappel: **Déterminisation d'un AFN (cas général)**

Étapes pour éliminer les transitions ε :

1. **Déterminer les ε-fermés (ou ε-closures) :**
   - L'ε-fermeture d'un état q, notée $\text{E-closure}(q)$, est l'ensemble des états atteignables depuis q en suivant uniquement des transitions ε.
   - On commence par inclure q dans son propre ε-fermeture. Ensuite, pour chaque état atteignable à partir de q par une chaîne de transitions ε, on ajoute cet état.
2. **Créer de nouvelles transitions :**
   - Pour chaque état q, et pour chaque symbole a de l'alphabet, déterminer tous les états atteignables à partir de $\text{E-closure}(q)$ en suivant une transition étiquetée a.
   - Ajouter une transition directe de q vers ces états pour le symbole a.
3. **Mettre à jour les états acceptants :**
   - Un état q devient un état final si au moins un état dans $\text{E-closure}(q)$ est un état final dans l'automate original.
4. **Supprimer les transitions ε :**
   - Une fois les nouvelles transitions ajoutées, on peut supprimer toutes les transitions ε de l'automate.

3. Analyse Lexicale Manuelle

Rappel: **Déterminisation d'un AFN (cas général)**

Algorithme:

Principe : considérer l'ε-fermeture des ensembles d'états

Définition : On appelle **ε-fermeture** de l'ensemble d'états $T = \{e_1, \ldots, e_n\}$ l'ensemble des états accessibles depuis un état e_i de T par des ε-transitions

Calcul de l'ε-fermeture de $T = \{e_1, \ldots, e_n\}$

```
Mettre tous les états de T dans une pile P
Initialiser ε-fermeture (T) à T
Tant que P est non vide faire
    Soit p l'état en sommet de P
    dépiler P
    Pour chaque état q tel qu'il y a une ε-transition entre p et q faire
        Si q n'est pas déja dans ε-fermeture (T)
            ajouter q à ε-fermeture (T)
            empiler q dans P
        finsi
    finpour
fin tantque
```

3. Analyse Lexicale Manuelle

Rappel: **Déterminisation d'un AFN (cas général)**

Algorithme:

Déterminisation :

1. Partir de l'ε-fermeture de l'état initial

2. Rajouter dans la table de transition toutes les ε-fermetures des nouveaux "états" produits, avec leurs transitions

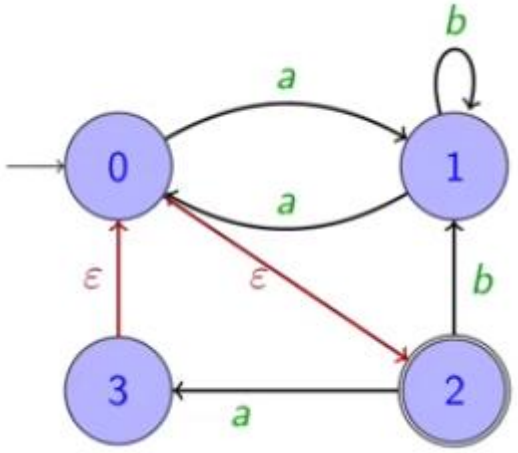
3. Recommencer 2 jusqu'à ce qu'il n'y ait plus de nouvel "état"

4. Tous les "états" contenant au moins un état terminal deviennent terminaux

5. Renuméroter alors les états.

3. Analyse Lexicale Manuelle

Rappel: Déterminisation d'un AFN (cas général)



	a	b	ε
→0	1	-	2
1	0	1	-
2	3	1	-
3	-	-	0

	a	b
→{0,2}	{1,3,2,0}	1
{1,3,2,0}	{1,3,2,0}	1
1	{0,2}	1

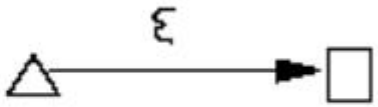
3. Analyse Lexicale Manuelle

Rappel: **Construction d'un AFN à partir d'une E.R**

Méthode 1: Construction de Thompson

(Construction de Thompson, basée sur la récursivité des expressions régulières)
Pour une expression régulière s, on note A(s) un automate reconnaissant cette expression.

automate acceptant la chaîne vide



automate acceptant la lettre *a*

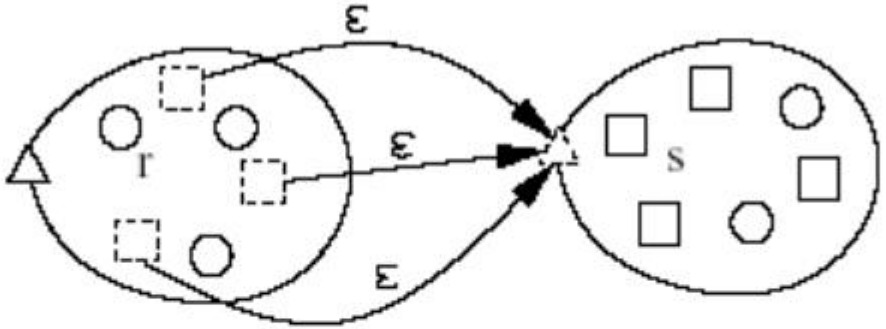
3. Analyse Lexicale Manuelle

Rappel: **Construction d'un AFN à partir d'une E.R**

Méthode 1: Construction de Thompson

(Construction de Thompson, basée sur la récursivité des expressions régulières)
Pour une expression régulière s, on note A(s) un automate reconnaissant cette

- automate acceptant $(r)(s)$



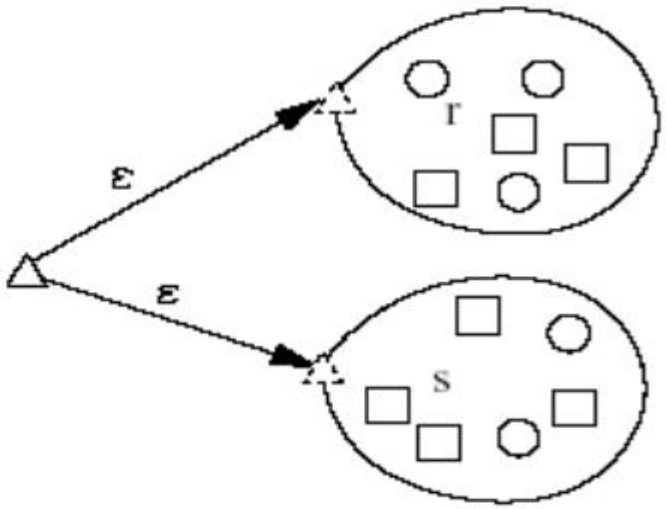
1. mettre une ε -transition de chaque état terminal de A(r) vers l'état initial de A(s)
2. les états terminaux de A(r) ne sont plus terminaux
3. le nouvel état initial est celui de A(r)
4. (l'ancien état initial de A(s) n'est plus état initial)

3. Analyse Lexicale Manuelle

Rappel: **Construction d'un AFN à partir d'une E.R**

Méthode 1: Construction de Thompson

- automate reconnaissant $r|s$



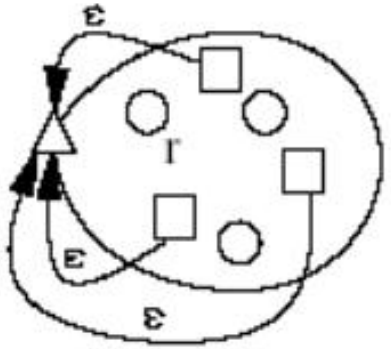
1. créer un nouvel état initial q
2. mettre une ε -transition de q vers les états initiaux de A(r) et A(s)
3. (les états initiaux de A(r) et A(s) ne sont plus états initiaux)

3. Analyse Lexicale Manuelle

Rappel: **Construction d'un AFN à partir d'une E.R**

Méthode 1: Construction de Thompson

- automate reconnaissant r^+



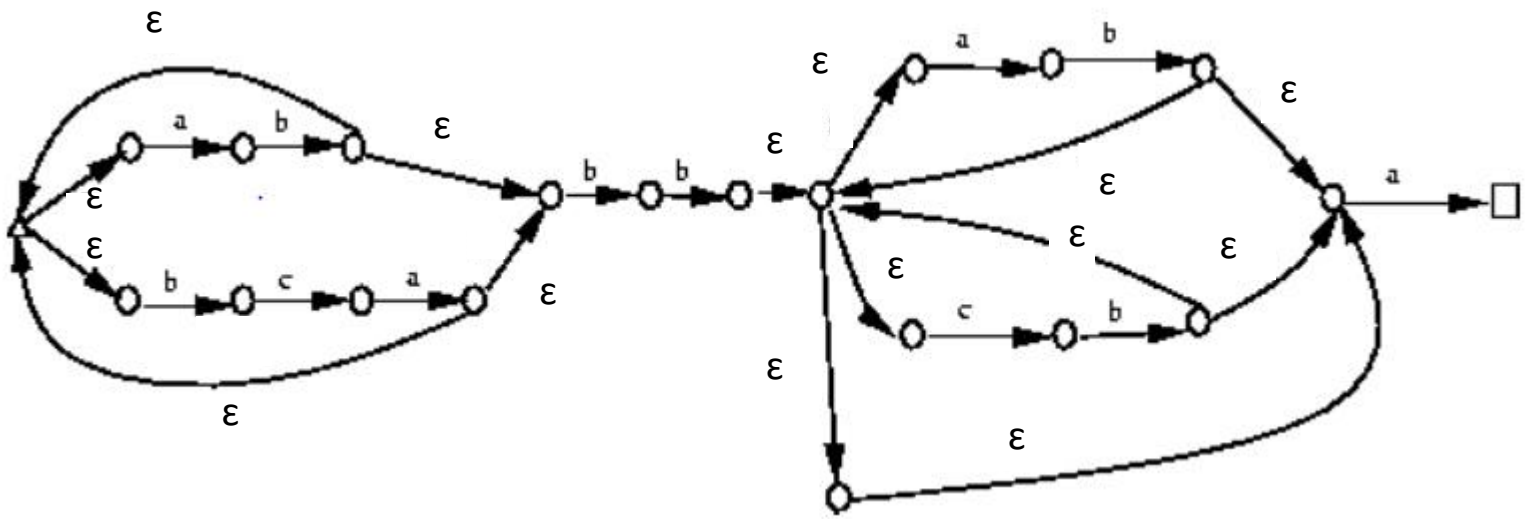
mettre des ε -transition de chaque état terminal de A(r) vers son état initial

3. Analyse Lexicale Manuelle

Rappel: **Construction d'un AFN à partir d'une E.R**

Méthode 1: Construction de Thompson

Exemple : $(ab|bca)^{+}bb(ab|cb)^{*}a$ correspond à l'automate:



3. Analyse Lexicale Manuelle

1. Identifier les différents types de jetons

```
if (x > 10) {
   y = 3.14;
   z = x + y;
}
```

1. Mot-clé : if
2. Identifiant : x, y, z
3. Nombre entier : 10
4. Nombre flottant : 3.14
5. Opérateurs : >, =, +
6. Ponctuation : (,), {, }, ;

3. Analyse Lexicale Manuelle

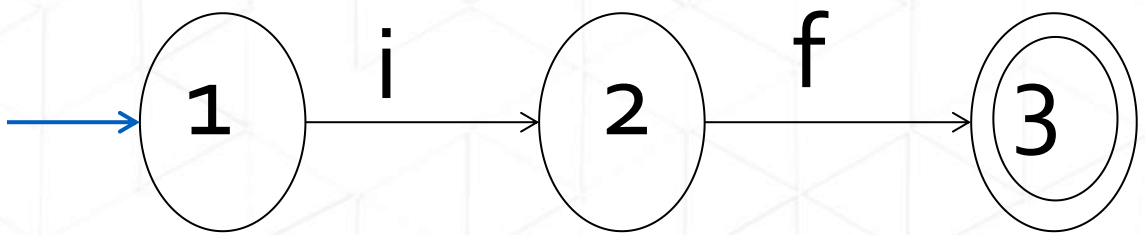
2. Définir les règles lexicales

```
if                                                    {return IF;}
[a - z] [a - z0 - 9 ] *                               {return ID;}
[0 - 9] +                                             {return NUM;}
([0 - 9] + “.” [0 - 9] *) | ([0 - 9] * “.” [0 - 9] +)   {return FLOAT;}
 (“--” [a - z]* “\n”) | (“ ” | “\n” | “\t”) +        {/* rien faire */}
.                                                     {error ();}
```

3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**
Dessiner l'automate

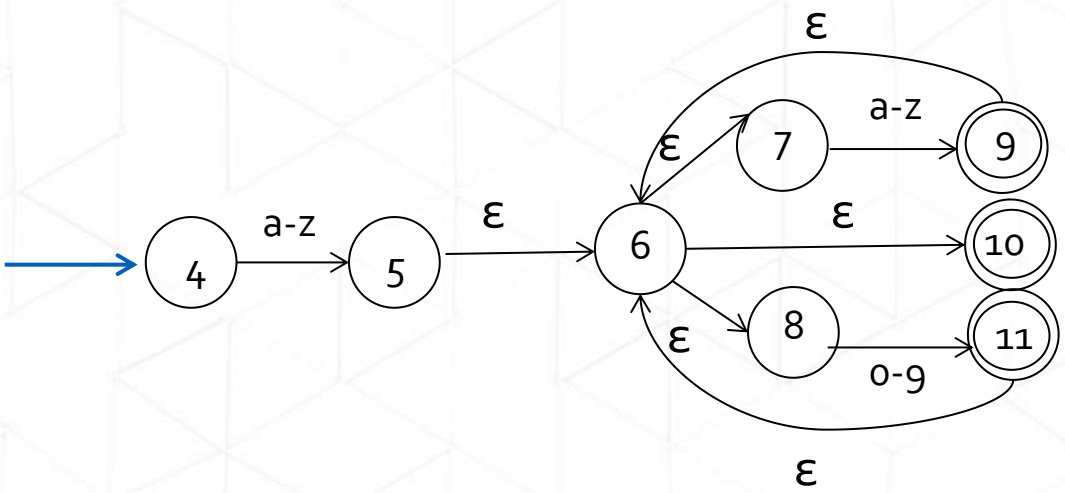
if	{return IF;}



3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**
Dessiner l'automate

[a - z] [a - z0 - 9] *	{return ID;}

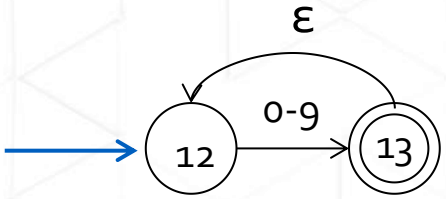


3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**

Dessiner l'automate

[0 - 9] +	{return NUM;}

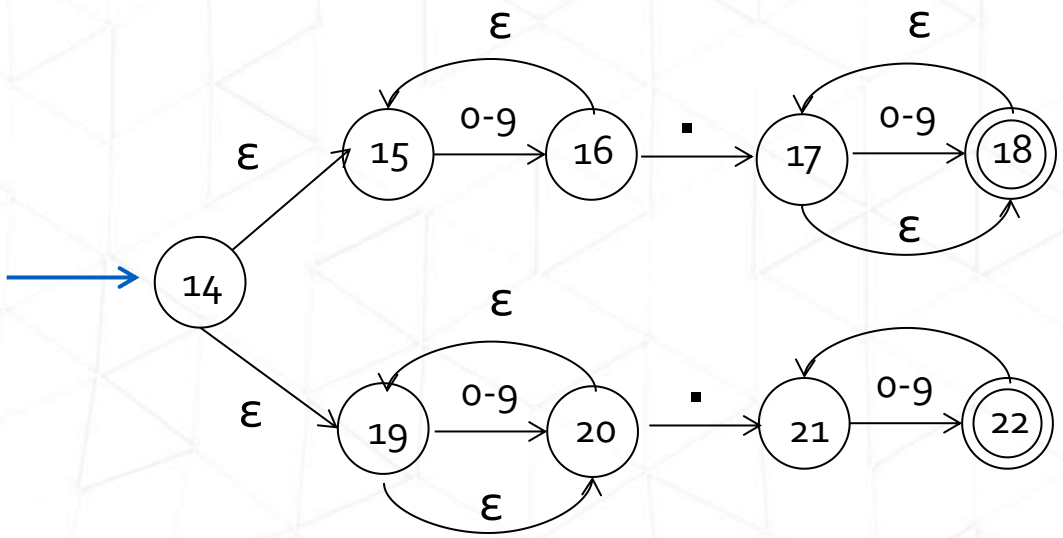


3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**

Dessiner l'automate

```
([0 - 9] + "." [0 - 9] *) | ([0 - 9] * "." [0 - 9] +)    {return FLOAT;}
```

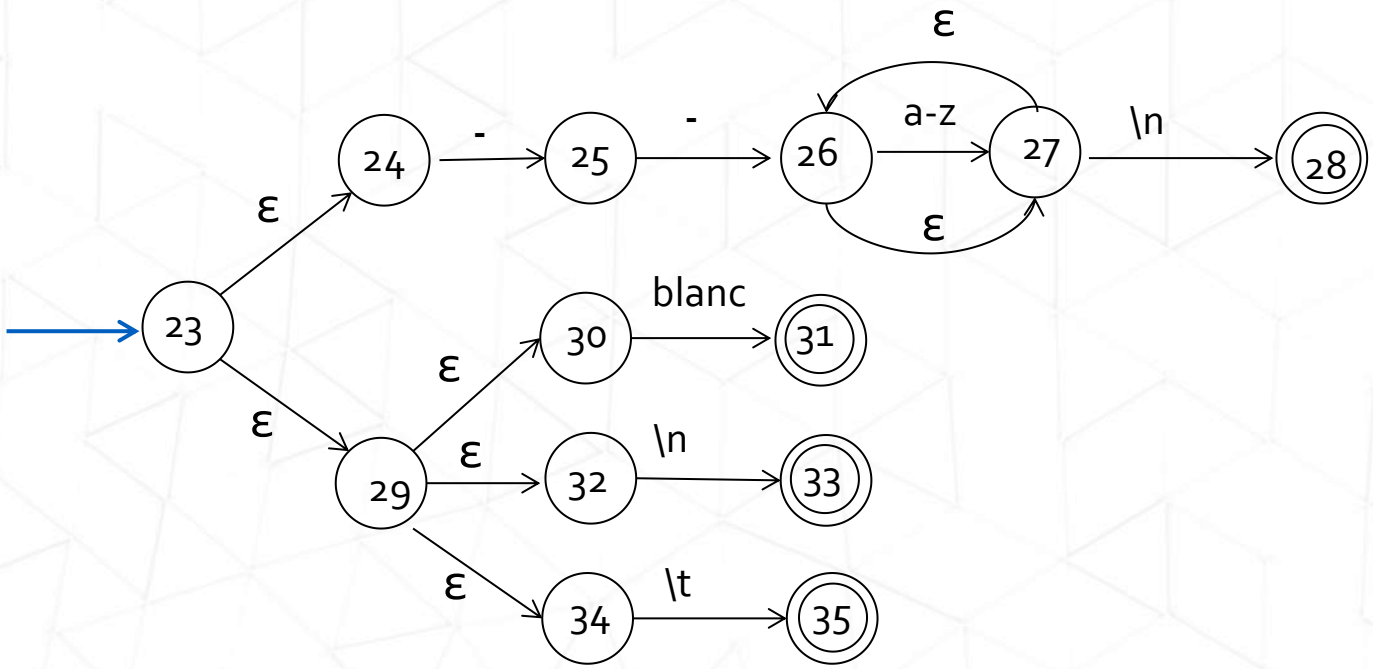


3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**

Dessiner l'automate

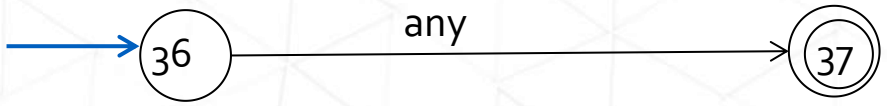
```
("--" [a - z]* "\n") | (" " | "\n" | "\t") +        {/* rien faire */}
```



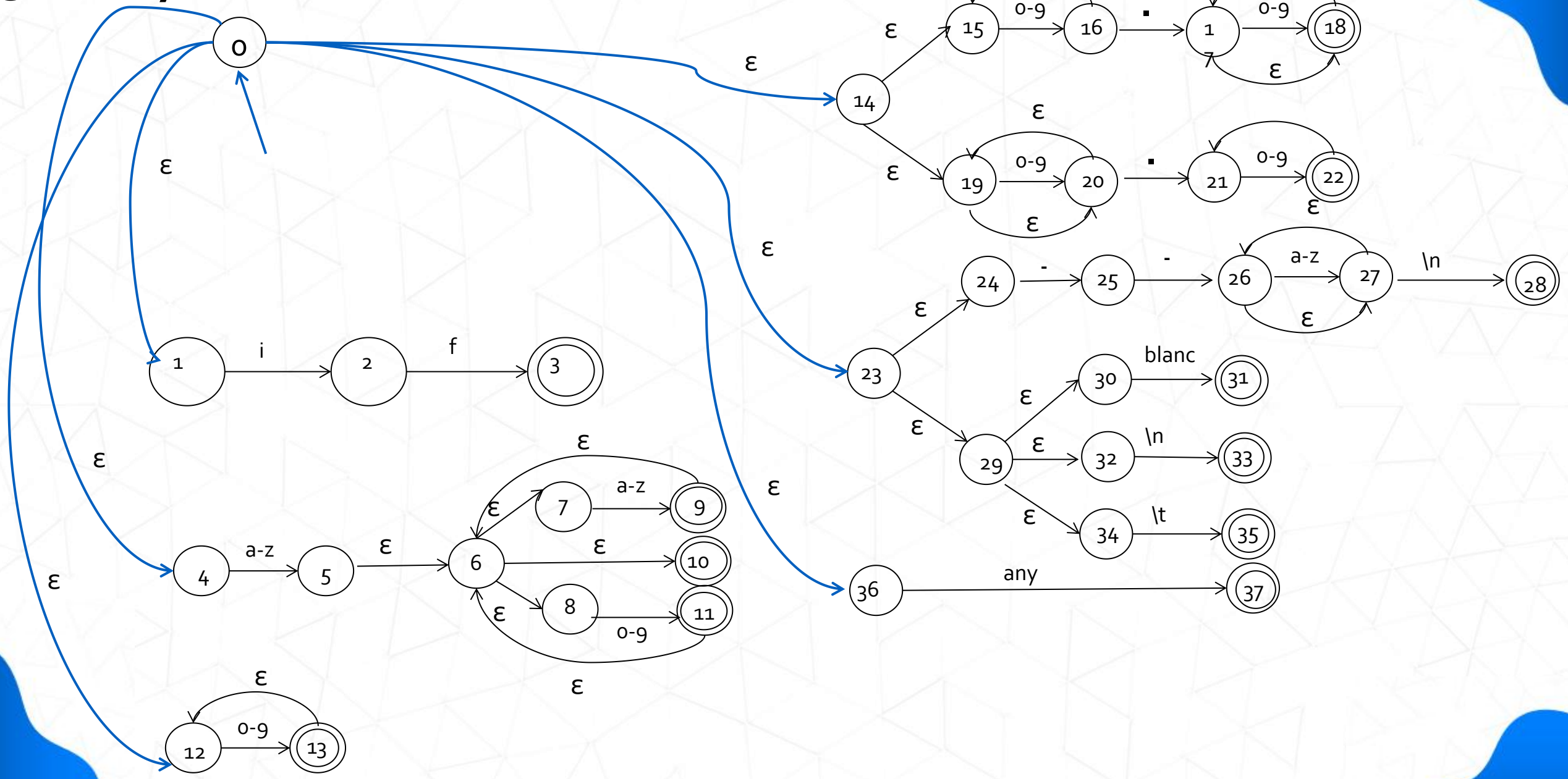
3. Analyse Lexicale Manuelle

3. **Implémenter les transitions de l'automate :**
Dessiner l'automate

.	{error ();}



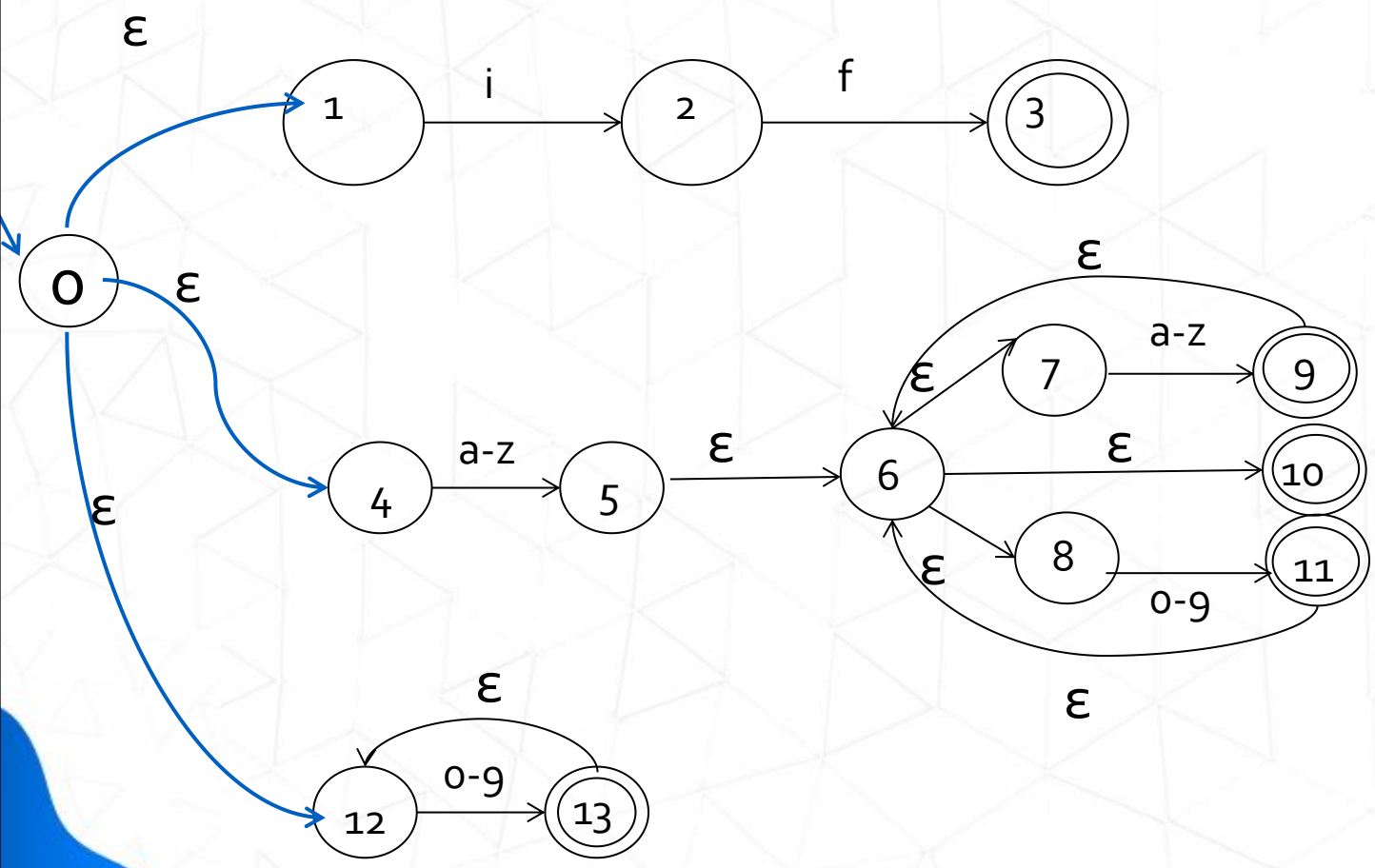
3. Analyse Lexicale Manuelle



3. Analyse Lexicale Manuelle

```
if                          {return IF;}
[a - z] [a - z0 - 9 ] *     {return ID;}
[0 - 9] +                   {return NUM;}
```

3. Implémenter les transitions de l'automate : Déterminisation



	i	f	a-z	0-9	ε
0	-1	-1	-1	-1	{1,4,12}
1	2	-1	-1	-1	-1
2	-1	3	-1	-1	-1
3	-1	-1	-1	-1	-1
4	-1	-1	5	-1	-1
5	-1	-1	-1	-1	6
6	-1	-1	-1	-1	{7,8,10}
7	-1	-1	9	-1	-1
8	-1	-1	-1	11	-1
9	-1	-1	-1	-1	6
10	-1	-1	-1	-1	-1
11	-1	-1	-1	-1	6
12	-1	-1	-1	13	-1
13	-1	-1	-1	-1	12